CompanyDocs RAG Assistant - Interview Study Guide

This guide is written for interview prep.
Use it to explain your project clearly, answer deep technical questions, and discuss tradeoffs honestly.

---

1) 60-second project pitch

I built a local RAG chatbot called **CompanyDocs RAG Assistant** using Python, FAISS, Sentence Transformers, and Gradio.
It answers questions from curated public GitLab docs/handbook pages.
The system ingests pages from a fixed URL list, cleans HTML, chunks text, creates embeddings, and stores vectors in FAISS.
At query time, it retrieves top relevant chunks and builds a grounded extractive answer with source citations.
I added guardrails to reject unsupported questions and created a custom evaluation suite that measures top-1/top-4 retrieval accuracy, citation coverage, and unsupported-query rejection accuracy.
I also compared a keyword baseline vs semantic retrieval and measured a +20 percentage-point top-4 improvement.

---

2) Resume-ready metrics you can state

From `evaluation/results.json`:

- `faiss_top_1_accuracy`: **0.28**
- `faiss_top_4_accuracy`: **0.56**
- `keyword_top_4_accuracy`: **0.36**
- `retrieval_improvement_points`: **20.0**
- `unsupported_query_rejection_accuracy`: **0.80**
- Retrieval latency (FAISS): around **0.01s** in your local run

How to say this in interview:

- "On a custom 35-question set, semantic retrieval reached 56% top-4 and improved by 20 points over a keyword baseline."
- "Unsupported-query rejection was 80%, which reduced unsupported answers."

Important honesty line:

- "This is a small custom benchmark and not a production-grade evaluation."

---

3) Architecture (explain in order)

1. `data/sources.txt`
   - Curated list of public documentation URLs (no broad crawling)
2. `scripts/ingest_docs.py`
   - Fetches pages, extracts clean text, chunks content

    - Outputs `data/chunks.json`
3. `scripts/build_index.py`
    - Embeds chunks with `all-MiniLM-L6-v2`
    - Builds FAISS index
    - Outputs `vector_store/index.faiss`, `vector_store/metadata.json`
4. `app.py`
    - Gradio UI for user questions
5. `rag/retriever.py`
    - Top-k vector search against FAISS
6. `rag/answer_builder.py`
    - Extractive grounded answer + fallback + citations
7. `evaluation/evaluate_retrieval.py`
    - Core retrieval and guardrail metrics
8. `evaluation/compare_baseline.py`
    - Keyword baseline vs FAISS comparison

---

4) File-by-file "what/how/why"

`scripts/ingest_docs.py`

- **What**: builds the text corpus from URLs.
- **How**: requests + BeautifulSoup + chunker; skips failed URLs with warnings.
- **Why**: deterministic ingestion from trusted docs; easy to debug and repeat.

`rag/text_cleaner.py`

- **What**: removes noisy HTML sections and normalizes text.
- **How**: strips script/style/nav/footer and compresses whitespace.
- **Why**: cleaner chunks improve retrieval signal.

`rag/chunker.py`

- **What**: splits text into ~500-word windows with overlap.
- **How**: sliding window over words; overlap = 80.
- **Why**: balances context preservation vs retrieval precision.

`scripts/build_index.py`

- **What**: generates embeddings and FAISS index.
- **How**: normalize embeddings, store in `IndexFlatIP`.
- **Why**: free local vector search with good speed and simple setup.

`rag/retriever.py`

- **What**: query embedding + top-k nearest chunks.
- **How**: same embedding model for docs and query; returns metadata + score.
- **Why**: shared reusable retrieval component for app and evaluation.

rag/answer_builder.py

- **What**: creates answer only from retrieved content.
- **How**: score threshold + supporting sentence selection by token overlap.
- **Why**: reduce hallucinations and keep answers evidence-grounded.

evaluation/evaluate_retrieval.py

- **What**: computes top-1/top-4, latency, citation coverage, rejection accuracy.
- **How**: loops through eval questions and checks expected source presence.
- **Why**: converts project quality into measurable numbers.

evaluation/compare_baseline.py

- **What**: compares semantic retrieval against keyword overlap.
- **How**: same eval set, separate retrieval methods, merged metrics.
- **Why**: demonstrates meaningful improvement, not just implementation.

---

5) Why these design choices (and tradeoffs)

Curated URL list

- **Pro**: reproducible, safe, small scope.
- **Con**: limited coverage.

Extractive answers (no heavy LLM generation)

- **Pro**: free hosting, lower hallucination risk, easy to explain.
- **Con**: less fluent synthesis.

FAISS local index

- **Pro**: fast and free.
- **Con**: no managed scaling or cloud reliability features.

Small custom evaluation

- **Pro**: practical and project-focused.
- **Con**: risk of sampling bias and overfitting to chosen questions.

---

6) Top interview questions and answer direction

Product/system

1. **Why RAG instead of fine-tuning?**
   Use case is dynamic docs QA; retrieval updates faster and cheaper than retraining.
2. **Why GitLab docs?**
   Real company-style documentation with policy/culture/process content.

3. **Why no chat memory?**
   First milestone prioritized grounded single-turn correctness and evaluation.

Data ingestion

4. **How do you handle 404/403 pages?**
   Script logs warnings and continues; avoids pipeline failure.
5. **How would you refresh data?**
   Schedule ingestion+index rebuild daily/weekly and version `chunks.json`.

Chunking/retrieval

6. **Why 500 words, overlap 80?**
   Enough context per chunk while still allowing precise retrieval.
7. **Why `all-MiniLM-L6-v2`?**
   Good quality/speed tradeoff and free local use.
8. **Why normalized vectors with `IndexFlatIP`?**
   Dot product on normalized vectors approximates cosine similarity.

Guardrails

9. **How do you reduce hallucinations?**
   Answer from retrieved chunks only; fallback for low-support queries.
10. **How did you set threshold 0.28?**
    Empirical heuristic from observed retrieval behavior; could tune on validation set.

Evaluation

11. **Why top-4 not just top-1?**
    RAG answers often combine evidence from several chunks.
12. **Why compare to keyword baseline?**
    Shows measurable gain from semantic retrieval.
13. **What does 80% rejection accuracy mean?**
    80% of unsupported queries were correctly rejected.

Scaling/production

14. **How would you scale to large corpora?**
    Switch to ANN index (IVF/HNSW), add reranker, caching, async ingestion.
15. **What would you monitor?**
    Latency, retrieval hit rate, citation coverage, fallback rate, error rate.

---

7) Mock interviewer cross-questioning

Q: "Your top-1 is 0.28. Isn't that low?"

Suggested response: "Top-1 is moderate, but top-4 is more relevant for RAG because multiple supporting chunks can still produce a valid grounded answer. I also measured baseline vs semantic improvement and saw

+20 points on top-4, which shows the retrieval approach is materially better than keyword search."

Q: "How trustworthy are these numbers?"

Suggested response: "I treat them as project-level metrics on a small custom set, not production claims. I call out this limitation and planned next steps are expanding the eval set and calibrating thresholds with held-out validation."

Q: "What would you improve first?"

Suggested response: "I'd first clean the source list to remove broken URLs, then add a reranker and threshold tuning. That should improve top-1 accuracy and citation coverage quickly."

---

8) Your strongest talking points

- You implemented full pipeline, not just UI.
- You worked under realistic cost constraints (free/local only).
- You added guardrails for unsupported questions.
- You measured outcomes and compared to a baseline.
- You deployed on Hugging Face Spaces with a live demo.

---

9) Weaknesses to acknowledge confidently

- Some source URLs returned 404/403, reducing corpus completeness.
- Evaluation set is small and manually curated.
- Answer generation is extractive, so wording can be rigid.
- No reranking step yet.

Use this sentence: "I focused on correctness, reproducibility, and explainability first; next iteration targets retrieval precision and broader evaluation."

---

10) Final resume bullets (ready to paste)

- Built a local RAG documentation assistant over curated GitLab handbook/docs pages using Python, FAISS, Sentence Transformers, and Gradio.
- Implemented end-to-end ingestion, cleaning, chunking, embedding, semantic retrieval, and source-cited grounded answering.
- Created a custom 35-question evaluation suite and measured 56% top-4 retrieval accuracy with 80% unsupported-query rejection accuracy.
- Benchmarked semantic retrieval against keyword search and improved top-4 accuracy by 20 percentage points.

---

11) Final revision checklist before interview

- Can you explain data flow without looking at code?
- Can you justify chunk size, model choice, and index choice?

- Can you explain each metric and why it matters?
- Can you state at least 3 limitations honestly?
- Can you describe 3 concrete next improvements?
- Can you demo live and answer "what happens on unsupported query?"

If all are yes, you are interview-ready for this project.